

The full JSON Web Token (JWT) authentication cycle is a **stateless process** involving user login, token issuance, and token-based authorization for subsequent requests. The cycle typically employs both a short-lived **access token** and a long-lived **refresh token** for enhanced security and a seamless user experience.

The JWT Authentication Cycle: Step-by-Step

Here is a breakdown of the typical steps in a modern JWT authentication system:

1. **User Login:** The user provides their credentials (e.g., username and password) to the authentication server via a secure (HTTPS) login request.
2. **Server Verification:** The server verifies the credentials against the data stored in its database. If the credentials are valid, the authentication succeeds.
3. **Token Generation:** Upon successful authentication, the server generates a JWT (access token) and a separate refresh token, both signed with a secret key known only to the server.

Access Token: Short-lived (e.g., 5-15 minutes) and contains user claims (ID, role, etc.).

Refresh Token: Long-lived (e.g., days or weeks) and used to obtain a new access token without re-authenticating.

4. **Token Transmission:** The server sends both tokens back to the client. The client stores the access token (e.g., in application memory or an HTTP-only cookie) and securely stores the refresh token (e.g., in a secure, HTTP-only cookie).
5. **Accessing Protected Resources:** For all subsequent requests to protected API endpoints, the client includes the access token, usually in the Authorization header as a Bearer <token>.
6. **Server Validation and Authorization:** The resource server intercepts the request and validates the JWT by:
 0. Checking its structure and format.
 1. Verifying the signature using its secret key to ensure the token hasn't been tampered with.
 2. Checking the expiration time and other claims (issuer, audience).
 3. If valid, the server grants access to the protected resource.
7. **Token Expiration and Refresh:** When the short-lived access token expires, the client uses the refresh token to send a request to a dedicated refresh endpoint on the authentication server.
8. **New Token Issuance:** The server verifies the refresh token (checking its validity and ensuring it hasn't been revoked or is not expired). If valid, the server issues a new pair of access and refresh tokens, and invalidates the old refresh token (token rotation). The client stores the new tokens and continues making requests seamlessly.
9. **Logout:** The user logs out, prompting the client to clear stored tokens (both access and refresh tokens). The server may also explicitly invalidate the refresh token on its end, often by adding it to a blacklist or revocation list, to prevent further use.